

DEVELOPER GUIDE



Building with the Amen API Integration Starter Kit

Everything a developer needs to go from an empty folder to a working, production-ready Amen integration: the required steps, the day-to-day workflow, and the best practices proven against the live sandbox.

Audience

Developers integrating the Amen Platform API

Repository

github.com/KMG411/amen-integration-starter-kit

Version

1.0 · API v1

CONTENTS

1. What this repository is	7. Webhooks
2. Set up your machine	8. Testing & continuous integration
3. Pick a stack & run the golden path	9. Security best practices
4. How the repository is organised	10. Go-live checklist
5. The integration workflow	Appendix A. Deal lifecycle & endpoints
6. Real API behaviour you must handle	Appendix B. Troubleshooting & support

01 What this repository is

The **Amen API Integration Starter Kit** is the official reference implementation of the Amen Platform API. It contains one small, working, copy-ready integration in every major technology stack, all built to the **same layout** and driven by the **same reference scenario** (a product escrow deal from creation to seller payout).

Use it as the model for your own integration instead of building from the raw API reference. Every client in the kit already handles the real-world details that are easy to miss — authentication, write protection, retries, timestamps and webhook verification — and every one of those behaviours has been confirmed against the live Amen sandbox.

Verified end to end. The reference scenario has been run in full against the real sandbox: a product deal was created, paid, executed, completed and paid out to the seller, and each step's webhook was received and its signature verified. The behaviours documented in section 6 and 7 are ground truth, not assumptions.

10

Stacks

Python, TypeScript, JavaScript, PHP, Java, C#, Kotlin, Dart/Flutter, Swift, plus a Postman collection.

1

Scenario

One golden path (`scenario/golden-path.yml`) that every stack runs identically.

MIT

Licence

Copy any part directly into your own proprietary codebase — no attribution required.

Repository	https://github.com/KMG411/amen-integration-starter-kit
API reference	https://docs.amnn.sa · spec vendored at openapi/openapi.yml
Sandbox API	https://sandbox-api.amnn.sa — safe test customers, deals and payments
Live API	https://api.amnn.sa — real money; separate token
Auth	header <code>X-API-Token: <token></code> on every request; tokens are environment-specific

02 Set up your machine

You need three things before writing any code: the repository, a sandbox token, and the toolchain for your chosen stack.

Step 1 — Get a sandbox token

1. Sign in to the sandbox dashboard: `https://platform-sandbox.amn.sa`.
2. Open **Integrations** → **Public API** and create a Public API account.
3. Tick **Enable REST API** for that account (without it, every call returns `403 api_access__disabled`).
4. Copy the **Secret key** — this is your `AMN_API_KEY`. It is shown once.

Step 2 — Clone and configure

```
# clone the reference kit
git clone git@github.com:KMG411/amen-integration-starter-kit.git
cd amen-integration-starter-kit

# create your local environment file
cp .env.example .env
```

Edit `.env` and set your token. The default base URL is the sandbox — leave it until you are ready for production.

```
AMN_API_KEY=your-sandbox-secret-key
AMN_ENV=sandbox
# AMN_WEBHOOK_SECRET is only needed by the webhook receiver (section 7)
```

One `.env` per project. Each stack loads the **nearest** `.env` only. Keep your token in the kit's own `.env`; never commit it. `.env` is already git-ignored, and CI runs a secret scan (gitleaks) on every push.

Step 3 — Install your stack's toolchain

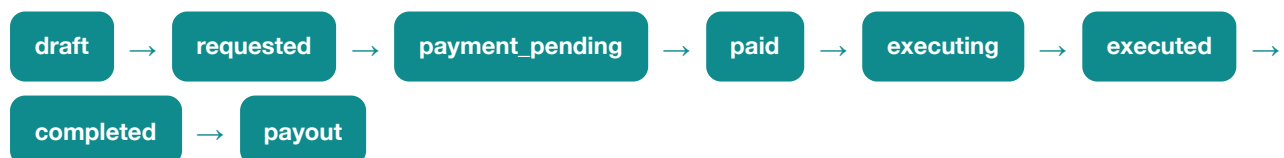
You only need the toolchain for the stack you are using — e.g. Python 3.11+, Node 20+, PHP 8.1+, JDK 17, .NET 8, Dart 3, or Swift 5.9. The exact install command per stack is in the next section.

03 Pick a stack & run the golden path

Start by running the reference scenario end to end. It proves your token works and shows the whole deal lifecycle before you write a line of your own code.

Stack	Folder	Install & run
Python 3.11	python/	<code>pip install -e ".[dev]" && python examples/01_golden_path.py</code>
TypeScript / Node 20	typescript/	<code>npm install && npm run example:golden-path</code>
JavaScript / Node 20	javascript/	<code>npm install && npm run example:golden-path</code>
PHP 8.1+	php/	<code>composer install && php examples/01_golden_path.php</code>
Java 17 (Maven)	java/	<code>mvn -q compile exec:java</code>
C# / .NET 8	csharp/	<code>dotnet run --project examples/GoldenPath</code>
Kotlin (Gradle)	kotlin/	<code>gradle :golden-path:run</code>
Flutter / Dart 3	flutter/	<code>cd amen_client && dart pub get && dart run example/golden_path.dart</code>
Swift 5.9	swift/	<code>swift run golden-path</code>
Postman	postman/	Import collection + <i>Amen</i> — <i>Sandbox</i> environment, set <code>apiToken</code> , run the folder in order

A successful run walks a product deal through every state and prints the status after each step:



```
environment: sandbox (https://sandbox-api.amnn.sa)
✓ customers IC-000000032 (buyer), IC-000000033 (seller)
✓ deal D-0000000002 created → status=draft
✓ submit → requested    ✓ approve → payment_pending
✓ pay with wallet → paid
✓ execution-start → executing    ✓ execution-complete → executed
✓ complete → completed    ✓ transfer-seller-amount → payout done
🎉 deal D-0000000002 finished: completed
```

If the run stops at `payment_pending` with a `NEEDS_TOP_UP` message, the sandbox wallet has no balance. Ask Amen support to top up the sandbox wallet, then re-run resuming from the deal number, e.g. `python examples/01_golden_path.py D-0000000002`.

04 How the repository is organised

Shared, language-neutral material lives at the root; each stack is a self-contained folder with the same internal shape.

Shared root

Path	Purpose
<code>openapi/openapi.yml</code>	Vendored official spec — the single source of truth for models
<code>openapi/error-catalogue.md</code>	Every error code, its meaning, and recommended handling (100 codes)
<code>scenario/golden-path.yml</code>	The reference scenario every stack runs identically
<code>docs/01-09</code>	Topic guides: auth, lifecycle, payments, webhooks, errors, security, versioning, troubleshooting, common mistakes
<code>docs/guide/</code>	This Developer Guide and the Partner Integration Guide (PDF + HTML source)
<code>.github/workflows/</code>	CI: unit tests, secret scan, nightly sandbox run, weekly spec-drift check

Every stack folder has the same layout

```
<stack>/
  src/ (or lib/, Sources/, main/)    the AmenClient library
  client                            configured HTTP client: auth, write protection, retries
  config                            reads AMN_API_KEY / AMN_ENV from the nearest .env
  models                            typed request/response objects from the spec
  errors                            AmenApiError with status + machine-readable codes
  resources/                         customers, deals, deal actions, lookups, webhooks
  webhooks/                         signature verification + framework-agnostic handler
  examples/                         01 golden path · 02 cancel/dispute · 03 withdrawal · 04 webhook receiver
  tests/                            unit tests (offline) + integration tests (sandbox)
```

Learn one, know them all. Because the shape is identical, once you understand the Python client you can read any other stack. Pick the reference stack closest to your own and mirror its structure.

05 The integration workflow

The recommended path from the reference kit to your own production integration:

- 1. Run the golden path** [proves your token + shows the lifecycle](#)
Run `examples/01_golden_path.py` (or your stack's equivalent) against the sandbox until it completes.
- 2. Copy the client into your codebase** [MIT — copy freely](#)
Copy the `src/amen` library (client, config, models, errors, resources, webhooks) into your project. It has few or no third-party dependencies by design.
- 3. Wire configuration to your secret store** [never hard-code tokens](#)
Replace the `.env` loader with however your app supplies secrets (environment variables, a vault, your framework's config). Keep the token server-side.
- 4. Build your business flow from the resource methods** [reuse, don't reinvent](#)
Compose `customers.create`, `deals.create`, `deals.actions.*` and the lookups into your product's flow. The lifecycle guard and retry logic come for free.
- 5. Add a webhook receiver** [section 7](#)
Expose one HTTPS endpoint, verify every delivery, respond `200` fast, and do the real work asynchronously.
- 6. Test against the sandbox, then switch to live** [section 10](#)
Point your integration tests at the sandbox, then flip `AMN_ENV` to `live` with a separate token when you go to production.

A minimal deal, in code

```
# Python – the same shape exists in every stack
amen = AmenClient() # reads AMN_API_KEY / AMN_ENV
buyer = amen.customers.create(first_name="Buyer", last_name="K", phone_code="SA",
                             phone_number=...)
seller = amen.customers.create(first_name="Seller", last_name="K", phone_code="SA",
                              phone_number=...)

deal = amen.deals.create(offer_type="product", offer_category=category_id,
                        offer_title="...", offer_price="100.00",
                        offer_delivery_fee="10.00") # must be > 0 – see section 6
amen.deals.set_parties(deal.number, buyers=[buyer.number], sellers=[seller.number])
amen.deals.set_delivery_address(deal.number, city=city_id, district="...", ...)

amen.deals.actions.submit(deal.number) # requested
amen.deals.actions.approve(deal.number) # payment_pending
amen.deals.actions.pay_with_wallet(deal.number) # paid (or pay_online for a checkout)
```

06 Real API behaviour you must handle

These behaviours are not obvious from the spec alone. They were confirmed by running real requests against the sandbox, and **every client in the kit already handles them for you** — this section explains what to preserve if you adapt the code.

Behaviour	What it means & what the client does
Write protection (CSRF)	The token alone is enough for <code>GET</code> , but <code>POST/PUT/DELETE</code> also need a double-submit token: a 32-hex value sent as both the <code>X-CSRFToken</code> header and the <code>csrftoken</code> cookie. The client generates one per instance and attaches it to every write.
Language header	Lookup endpoints (<code>categories</code> , <code>cities</code> , <code>dispute-reasons</code> , <code>cancel-reasons</code>) return <code>500</code> with no <code>Accept-Language</code> header. The client always sends <code>Accept-Language: en</code> .
Timestamps	Deal <code>created_at</code> / <code>updated_at</code> arrive as ISO-8601 strings from the REST API but as epoch-milliseconds inside webhook bodies. Parse defensively; don't assume one format.
Delivery fee > 0	Creating a product deal with <code>offer_delivery_fee: "0.00"</code> is rejected as <code>offer_delivery_fee__required</code> — a zero fee is treated as missing. Send a positive value.
Money is a string	All amounts are two-decimal strings in SAR (<code>"100.00"</code>). Never send floats.
Auth error codes	A wrong environment or bad token returns <code>403 api_token__invalid</code> (not 401); an account without REST enabled returns <code>403 api_access__disabled</code> .

If you write your own HTTP layer instead of copying a client, the write-protection token is the detail most likely to bite you: a plain `POST` with only the API token returns `403 "CSRF cookie not set"`. Some HTTP libraries also strip manually-set cookies — the kit disables that where needed (for example .NET `UseCookies=false`, Swift `httpShouldSetCookies=false`).

07 Webhooks

Amen notifies your server of deal events by `POST` ing to a URL you register. Register once and store the secret it returns:

```
POST /api/v1/web-hooks/ { "url": "https://your-app/webhooks/amen" }
→ { id, url, secret_key } # secret_key is shown once – store as AMN_WEBHOOK_SECRET
```

The signature scheme (verified against live deliveries)

Every delivery is signed. Amen signs the **timestamp and the raw body together**, Stripe-style — not the body alone:

```
signed_message = "{X-Webhook-Timestamp}" + "." + raw_body
X-Webhook-Signature: sha256=HMAC_SHA256(AMN_WEBHOOK_SECRET, signed_message)
```

Header	Meaning
X-Webhook-Event	Event type, e.g. <code>deal.status.changed</code> (also present as <code>event</code> in the body)
X-Webhook-Signature	HMAC-SHA256 hex, <code>sha256=</code> prefixed
X-Webhook-Timestamp	ISO-8601; part of the signed message — read it from the header

The body envelope is `{ event, timestamp, payload }`, where `payload` is the full deal object; read `payload.status` for the new state.

Receiver rules

1. Read the **raw** request body before parsing — re-serialising the JSON changes the bytes and breaks the signature.
2. Verify the signature over `timestamp + "." + raw_body` with your secret, in constant time. Reject on mismatch.
3. Respond `200` quickly; queue the real work asynchronously.
4. De-duplicate — there is no event id, so key on the event `timestamp` (unique per delivery). Deliveries can repeat.
5. Never trust the payload alone for money decisions — re-read the deal via `GET /deals/{number}`.

The kit's `webhooks/` module implements all five rules; you pass it the request headers and raw body and handle the returned event.

08 Testing & continuous integration

Unit tests (offline)

Every stack ships fast unit tests that mock the network — encoding, error mapping, retry logic and webhook signatures (each asserts the same golden signature vector). Run them before every commit.

Integration tests (sandbox)

Mirror `scenario/golden-path.yml` and hit the real sandbox. They run only when `AMN_API_KEY` is set, so they never break an offline build.

What CI runs on every push

Workflow	Purpose
<code>ci</code>	Unit tests for all 10 stacks in parallel, plus a gitleaks secret scan
<code>sandbox-nightly</code>	Runs the golden path against the live sandbox each night to catch drift
<code>spec-drift</code>	Weekly check that the vendored spec still matches the published API

Before you open a pull request: run your stack's unit tests locally, keep the shared layout, and if you touch a client behaviour, update the matching test so the golden vector still passes.

09 Security best practices

- **Server-side only.** The API token grants full account access. Never ship it in a mobile app, browser bundle, or public repo. Mobile and web front-ends must call **your** backend, which holds the token and calls Amen.
- **Use the proxy pattern for apps.** The Flutter, Swift and Kotlin stacks include notes and a sample showing the client talking to your backend, not directly to Amen. Follow it.
- **Separate tokens per environment.** Sandbox and live tokens are different and not interchangeable; a mismatch returns `403 api_token__invalid`. Store them in a secret manager, not in code.
- **Verify every webhook.** Treat an unverified delivery as hostile. Always check the signature (section 7) before acting.
- **Re-read before money moves.** Confirm state with `GET /deals/{number}` rather than trusting a webhook payload or a stale local copy.
- **Keep secrets out of git.** `.env` is ignored and CI scans for leaks — don't work around either.

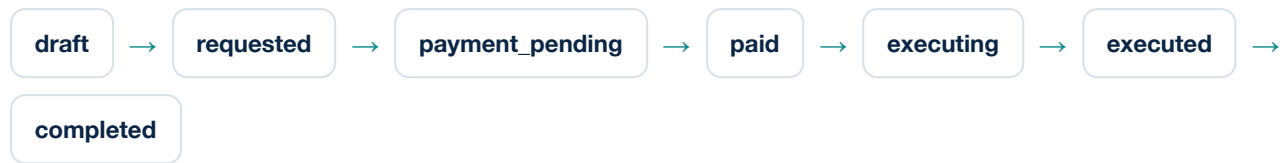
10 Go-live checklist

- ☐ Golden path completes end to end in the sandbox for your stack.
- ☐ Your own deal flow works against the sandbox, including cancel and dispute paths.
- ☐ Token is loaded from a secret manager, server-side only — never in client code or git.
- ☐ A live token has been issued (separate from sandbox) and REST API is enabled for it.
- ☐ Webhook endpoint is public over HTTPS, verifies signatures, and responds `200` quickly.
- ☐ Webhook processing is idempotent (de-duplicates on the event timestamp).
- ☐ Money decisions re-read the deal via `GET /deals/{number}`.
- ☐ Errors are handled by code (see `openapi/error-catalogue.md`), with retries on `429` / `5xx`.
- ☐ `AMN_ENV` switched to `live`; integration tests pass against the sandbox in CI.
- ☐ Monitoring/alerting is in place for failed payments, disputes and webhook errors.

When every box is ticked, switch `AMN_ENV` to `live`, deploy, and process a small real deal end to end as a final smoke test before opening the flow to customers.

A Appendix — Deal lifecycle & endpoints

Lifecycle states



Side states: `disputed`, `dispute_approved`, `canceled`, `payment_overdue`.

Core endpoints

Endpoint	Use
<code>POST /customers/</code>	Create buyer / seller
<code>POST /deals/</code>	Create a deal (needs a valid <code>offer_category</code> and delivery fee > 0)
<code>GET /categories/</code> , <code>GET /cities</code>	Lookups (send <code>Accept-Language: en</code>)
<code>POST /deals/{n}/action/{name}</code>	submit, approve, make-payment-wallet, execution-start/complete, complete, transfer-seller-amount, cancel, dispute...
<code>GET /deals/{n}</code>	Authoritative deal state — re-read before money moves
<code>POST /web-hooks/</code>	Register a webhook URL; returns the signing secret once

B Appendix — Troubleshooting & support

Symptom	Cause & fix
<code>403 api_token__invalid</code> on every call	Sandbox token used against live URL (or vice-versa), or a typo. Check the base URL printed at startup; re-copy the token.
<code>403 api_access__disabled</code>	REST API not enabled for the account — tick Enable REST API in the dashboard.
<code>403 "CSRF cookie not set"</code> on writes	Missing the double-submit token — use the kit's client, or send <code>X-CSRFToken</code> + <code>csrftoken</code> cookie (32 hex, identical).
<code>500</code> on categories/cities	Missing <code>Accept-Language</code> header — send <code>Accept-Language: en</code> .
<code>offer_delivery_fee__required</code>	You sent <code>"0.00"</code> — use a positive delivery fee.
Run stops at <code>payment_pending</code>	Sandbox wallet empty — ask Amen to top it up, then resume from the deal number.
Webhook signature never matches	You signed the body only, or re-serialised it. Sign <code>timestamp + "." + raw_body</code> over the exact received bytes.

More detail lives in `docs/08-troubleshooting.md` and the full `openapi/error-catalogue.md`.

Repository	<code>https://github.com/KMG411/amen-integration-starter-kit</code> — open an issue for anything unclear
API docs	<code>https://docs.amnn.sa</code>
Amen support	<code>info@mtahd.sa</code>

Amen API Integration Starter Kit · Developer Guide · Version 1.0 (API v1). Behaviours in sections 6–7 were verified against the live Amen sandbox.